



Lab Project Report

Only for course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	
Understanding/Analysis						
Implementation						
Accuracy						
Task Efficiency						
Total obtained mark						
Comments						

Semester:January -June 2026

Student Name:Amena Musharrat

Student ID:250701102

Batch:45

Section:C

Course Code: CSE124

Course Name: Object Oriented Programming

Course Teacher Name: Debabrata Mallick

Designation:Lecturer

Submission Date: 18/06/2026

No.	Contents	Page No.
1	Introduction & Project Overview	3
2	Project Objectives	4
3	Key Principles	5
4	System Architecture & Design	6
5	Codes of project	7
6	Output of the project	8
7	Functional Requirements	9
8	Real Life Comparison	10
9	Challenges & Future Scope	11

Introduction:

In today's digital world, banking systems are highly automated to provide faster and easier services. An Automated Teller Machine (ATM) enables users to perform basic banking tasks such as cash withdrawal and balance inquiry without needing to visit a bank.

Also,

- 1.The system ensures secure user authentication using Login-Registration method before accessing any banking features.
- 2.It provides a user-friendly interface (GUI/Console) that simulates real-world ATM operations for better understanding and usability.

INSERT CARD / LOGIN

Username :

PIN :

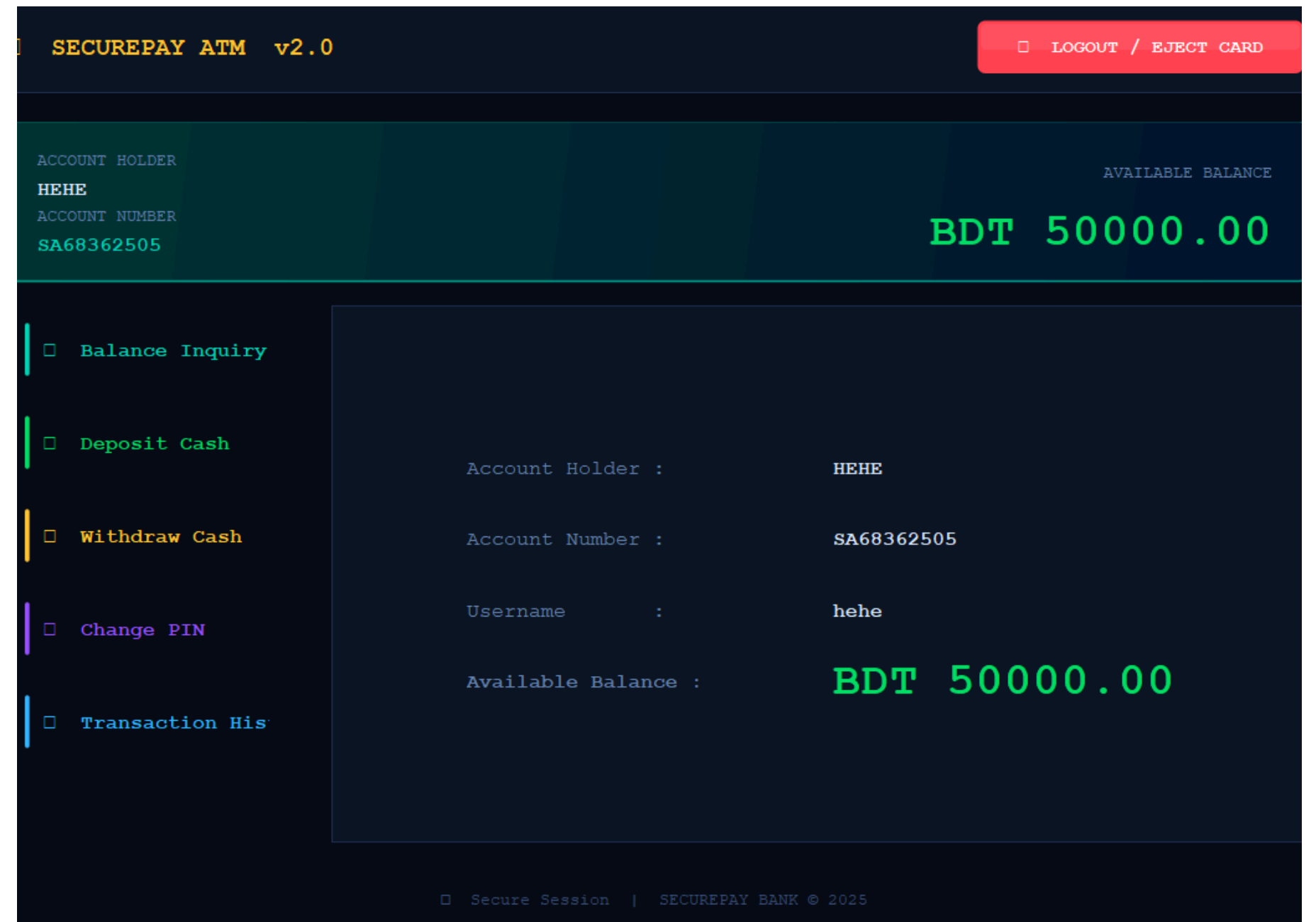
☐ LOGIN

☐ CREATE NEW ACCOUNT

Demo: register an account to begin

Project Objectives:

- a) To apply Java OOP concepts in developing a real-world expense management application.
- b) To ensure data integrity and maintain modular, reusable code for better system organization.
- c) To enable users to record and manage their incomes and expenses efficiently.
- d) To provide features for adding, editing, deleting and viewing transactions in categorized formats.
- e) To implement file handling for reliable and persistent data storage.
- f) To incorporate exception handling to enhance system stability.



Key OOP Principles

Encapsulation: Protects sensitive data by keeping account information private and allowing controlled access through methods

.

Abstraction: Hides complex internal processes and provides simple interfaces for users to perform ATM operations.

Inheritance: Enables code reusability by allowing classes to inherit common properties and behaviors from parent classes.

Polymorphism: Allows similar methods to perform different actions depending on the transaction type or context.

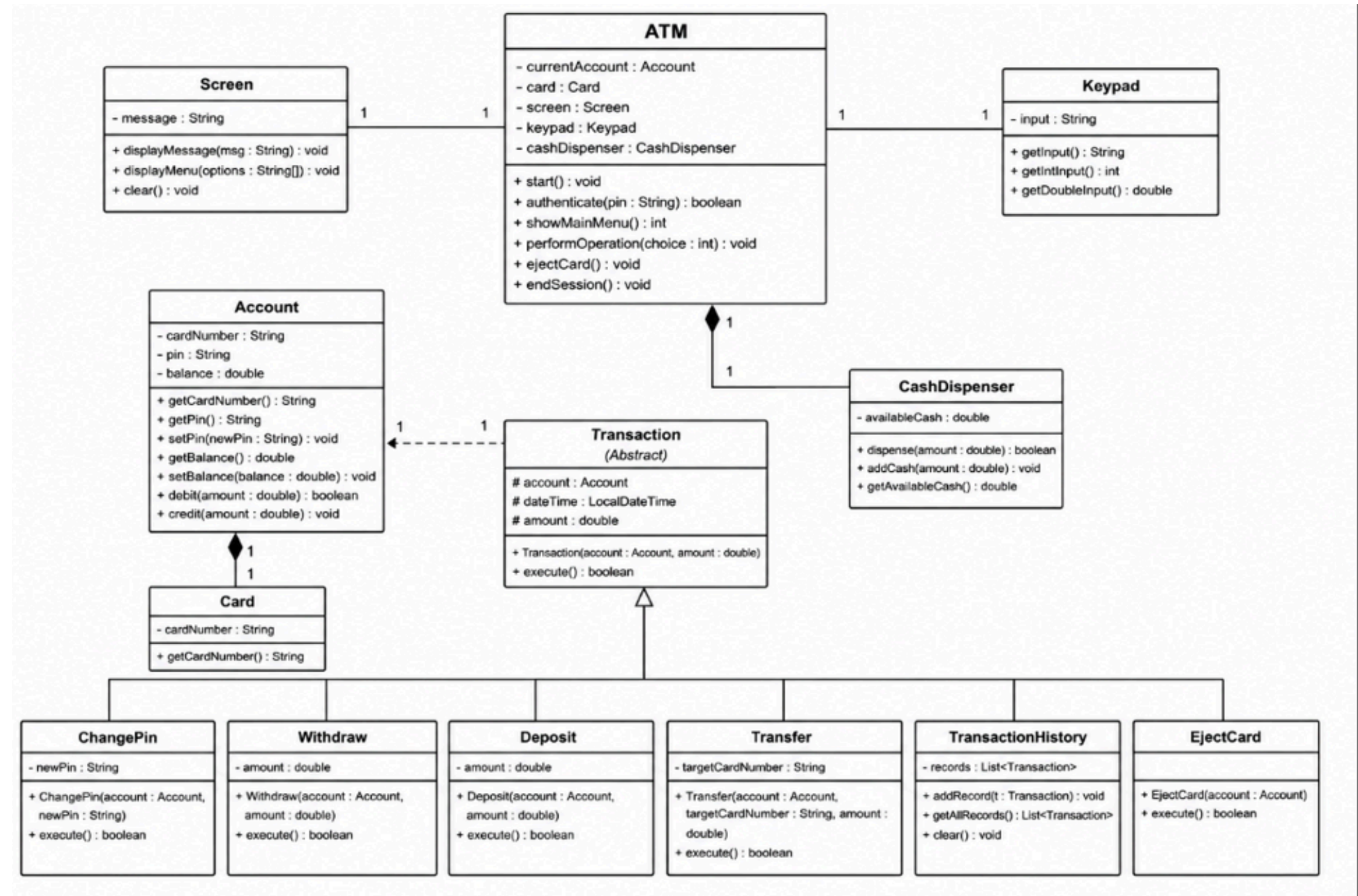
Modularity: Divides the system into separate classes and functions to improve maintainability and debugging.

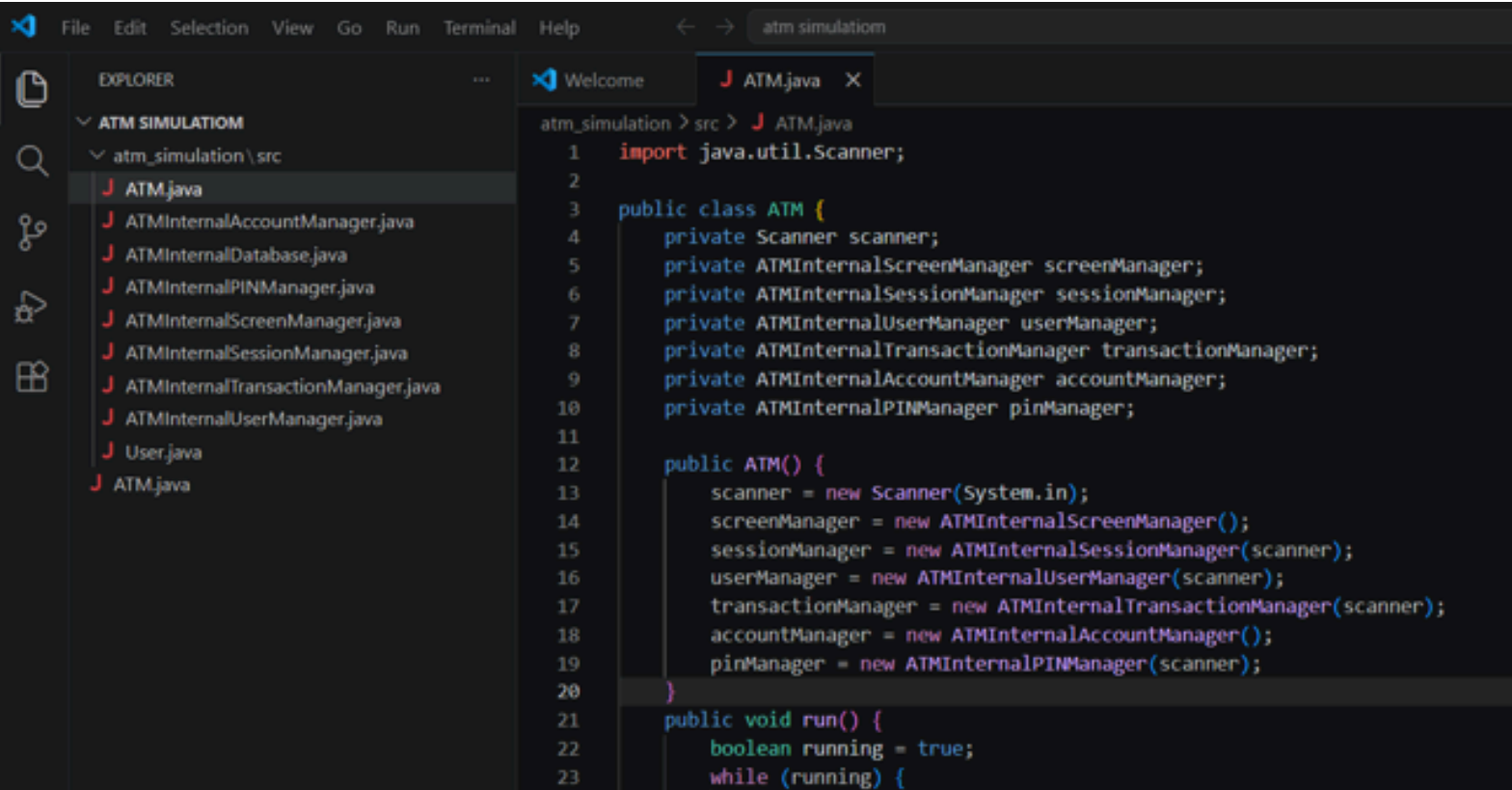
UML Class Diagram

The UML Class Diagram represents the structure of the ATM system by showing classes, attributes, methods, and relationships between them.

Main classes include:

- 1.ATM
- 2.Account
- 3.Bank Database
- 4.Transaction
- 5.Withdrawal
- 6.Deposit
- 7.Balance Inquiry
- 8.Screen
- 9.Keypad

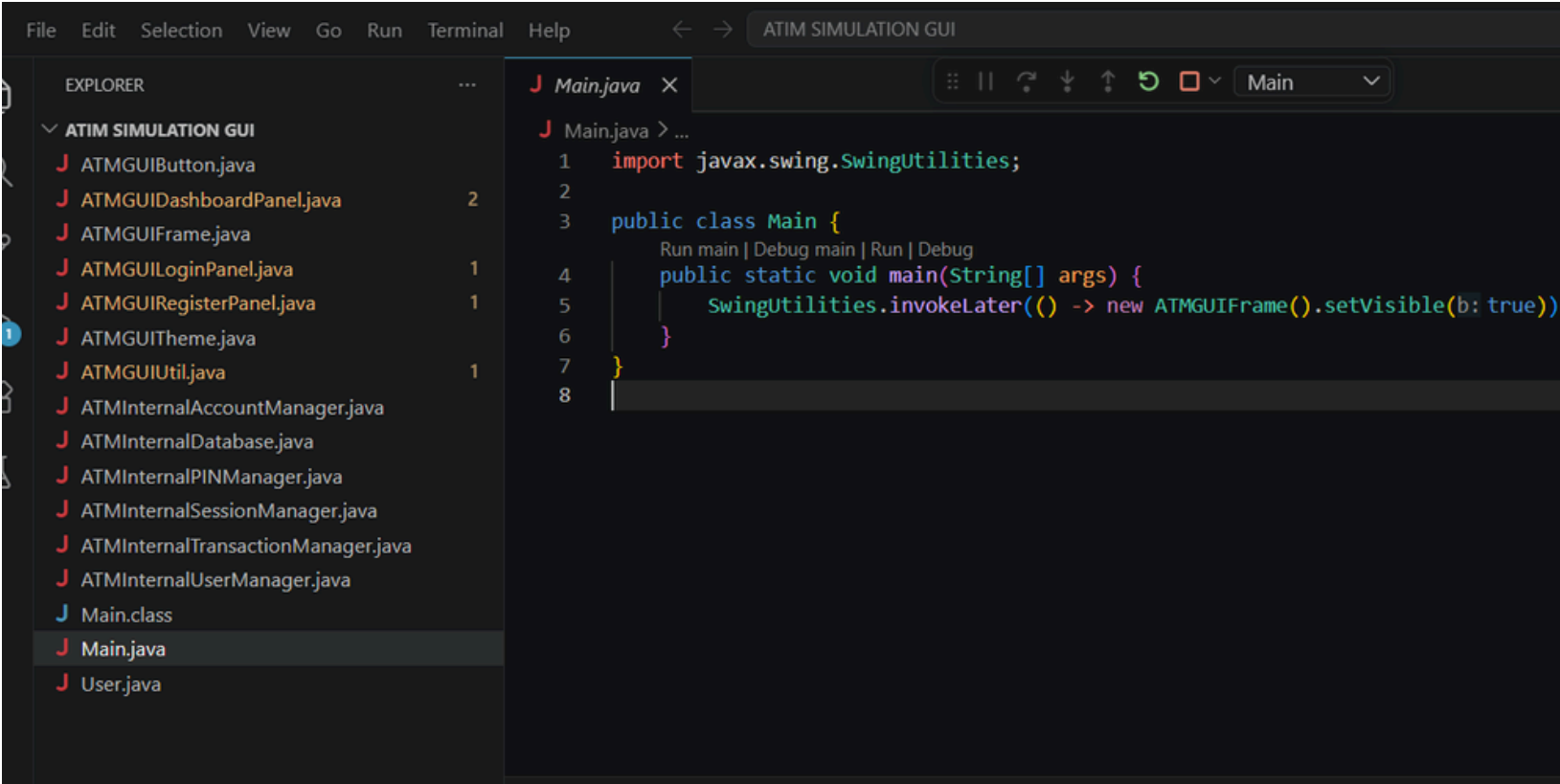




```
1 import java.util.Scanner;
2
3 public class ATM {
4     private Scanner scanner;
5     private ATMInternalScreenManager screenManager;
6     private ATMInternalSessionManager sessionManager;
7     private ATMInternalUserManager userManager;
8     private ATMInternalTransactionManager transactionManager;
9     private ATMInternalAccountManager accountManager;
10    private ATMInternalPINManager pinManager;
11
12    public ATM() {
13        scanner = new Scanner(System.in);
14        screenManager = new ATMInternalScreenManager();
15        sessionManager = new ATMInternalSessionManager(scanner);
16        userManager = new ATMInternalUserManager(scanner);
17        transactionManager = new ATMInternalTransactionManager(scanner);
18        accountManager = new ATMInternalAccountManager();
19        pinManager = new ATMInternalPINManager(scanner);
20    }
21    public void run() {
22        boolean running = true;
23        while (running) {
```

Explanation:

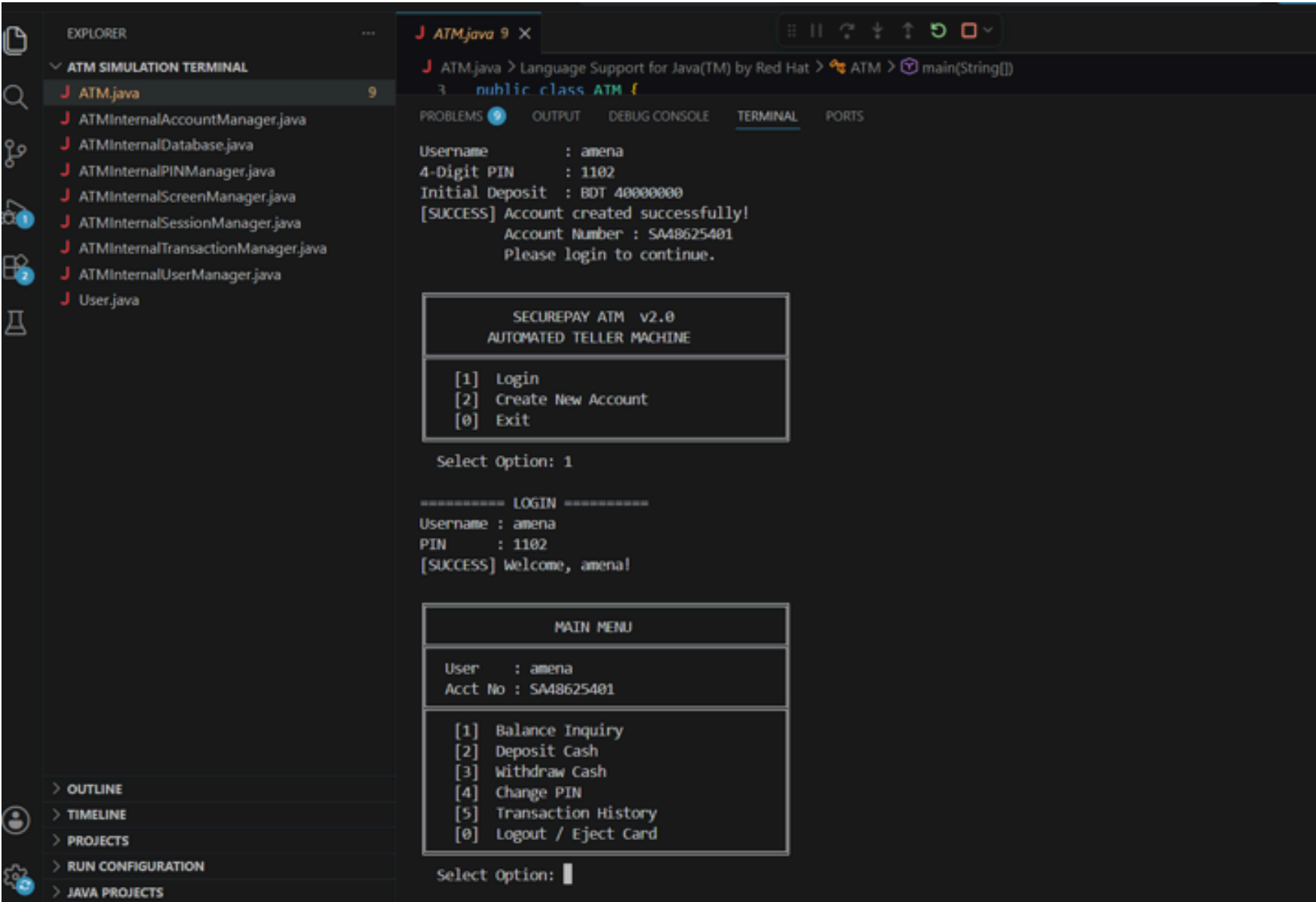
- 1. The ATM class is the main controller that manages all ATM operations and components.
It uses Scanner to take user input from the console.
Multiple manager classes (session, user, account, PIN, transaction, screen) are declared for modular design.
- 4. The constructor initializes all managers and connects them with the shared scanner.
- 5. The run () method contains the main loop that keeps the ATM system active.
- 6.It shows the welcome screen and takes user choice as input in each iteration.
- 7. A switch statement handles actions like login (case 1) and user registration (case 2).



```
1 import javax.swing.SwingUtilities;
2
3 public class Main {
4     Run main | Debug main | Run | Debug
5     public static void main(String[] args) {
6         SwingUtilities.invokeLater(() -> new ATMGUIFrame().setVisible(b: true));
7     }
8 }
```

Explanation:

- 1. import javax.swing.SwingUtilities; – Imports the SwingUtilities class to handle GUI operations safely in Java Swing.
- 2. public class Main – Creates the main class that contains the starting point of the program.
- 3. public static void main(String[] args) – Defines the main method where program execution begins.
- 4. SwingUtilities.invokeLater() – Runs the GUI on the Event Dispatch Thread (EDT) for smooth and safe execution.
- 5. () -> new ATMGUIFrame().setVisible(true) – Creates an ATM GUI frame object using a lambda expression and makes it visible.
- 6. new ATMGUIFrame().setVisible(true); – Opens and displays the ATM Simulation GUI window when the program starts.



```
ATM.java 9 X
J ATM.java > Language Support for Java(TM) by Red Hat > ATM > main(String[])
3 public class ATM {

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Username      : amena
4-Digit PIN   : 1102
Initial Deposit : BDT 40000000
[SUCCESS] Account created successfully!
           Account Number : SA48625401
           Please login to continue.

SECUREPAY ATM v2.0
AUTOMATED TELLER MACHINE

[1] Login
[2] Create New Account
[0] Exit

Select Option: 1

===== LOGIN =====
Username : amena
PIN      : 1102
[SUCCESS] Welcome, amena!

MAIN MENU

User      : amena
Acct No   : SA48625401

[1] Balance Inquiry
[2] Deposit Cash
[3] Withdraw Cash
[4] Change PIN
[5] Transaction History
[0] Logout / Eject Card

Select Option: 1
```

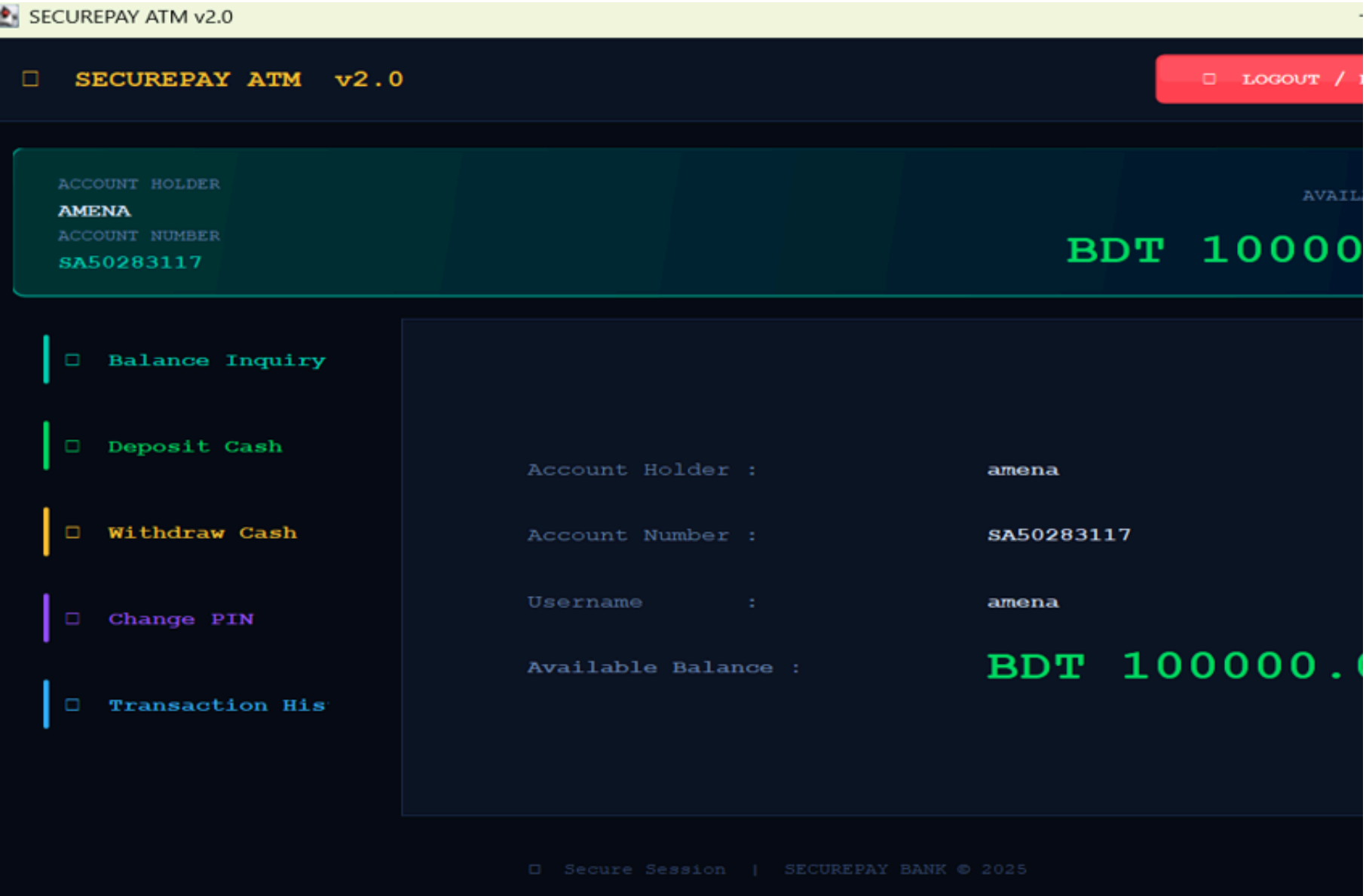
Explanation:

The ATM simulation output provides realistic banking experience through an interactive user interface and system responses.

The long screen displays account information, menus, and transaction options for user interaction.

The feature screen presents different ATM functionalities such as withdrawal, deposit, and balance checking.

The terminal output shows backend processing, user inputs, and transaction execution results.



GUI Explanation:

- Login System – Provides secure access through account number and PIN verification.
- Deposit Feature – Allows users to add money to their account balance.
- Withdrawal Feature – Enables users to withdraw money while checking balance availability.
- Balance Inquiry – Displays the current account balance instantly.
- Transaction History – Records and displays previous banking transactions..

Functional Requirements

The ATM Simulation System should perform the following functions:

User Registration and Login

Users can create an account with necessary information.

Existing users can log in using account number and PIN.

PIN Verification

The system verifies the entered PIN before granting access.

Invalid PIN attempts are restricted for security purposes.

Balance Inquiry

Users can check their current account balance.

Cash Withdrawal

Users can withdraw money if sufficient balance is available.

The system updates the remaining balance automatically.

Deposit Money

Users can deposit money into their account.

The balance is updated after successful deposit.

Fund Transfer

Users can transfer money from one account to another.

PIN Change

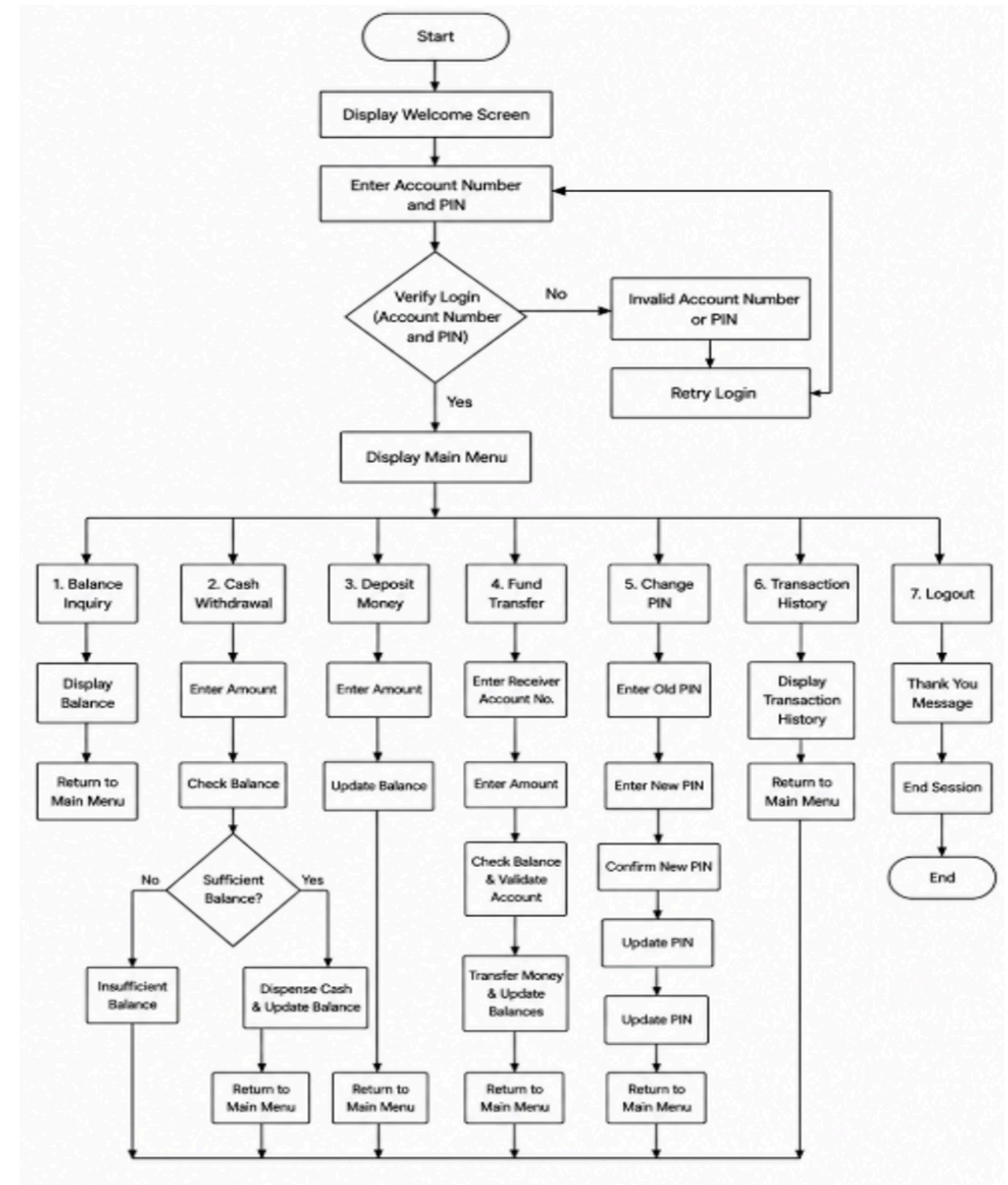
Users can securely change their ATM PIN.

Transaction History

The system stores and displays previous transaction records.

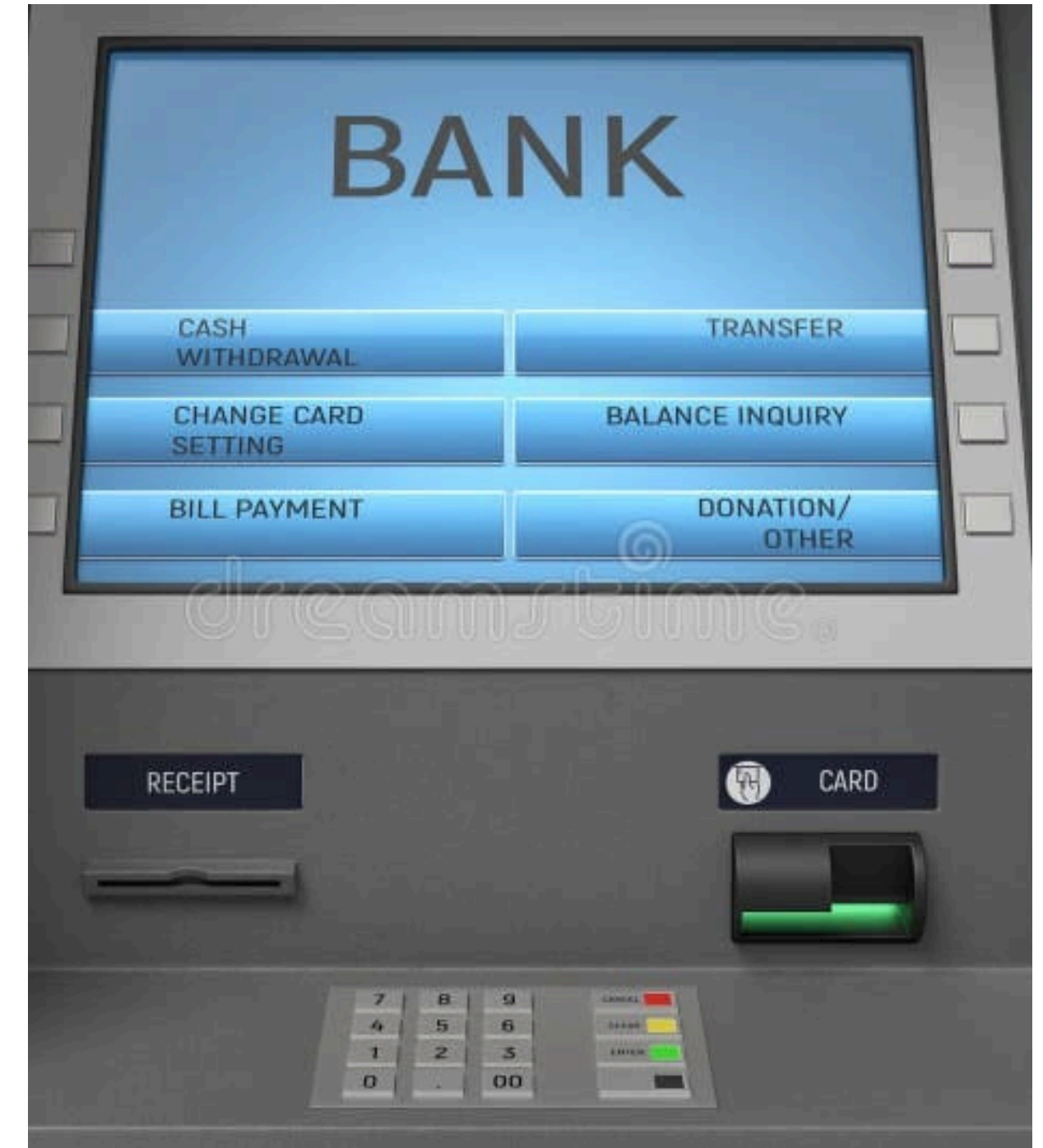
Logout Feature

Users can safely end their ATM session after completing transactions.



Real-Life Comparison:

- **Login System** – Similar to inserting an ATM card and entering a PIN before accessing banking services.
- **Deposit Feature** – Works like depositing cash into a real ATM machine.
- **Withdrawal Feature** – Similar to withdrawing money from a physical ATM.
 - **Balance Inquiry** – Functions like checking account balance through a real ATM screen.
 - **Transaction History** – Similar to viewing or printing transaction records from an ATM.
 - **PIN Change Option** – Works like changing an ATM card PIN for security purposes.
- **User-Friendly Interface** – Simulates the screen-based interaction experience of a real ATM machine.



Challenges & Future Scope

Limitations

The system works only in a simulated environment without real banking integration.

Limited security features compared to real ATM systems.

Does not support multi-user database storage (if not implemented).

Future Enhancements

Integration with a database for real account management.

Implementation of advanced security features like OTP verification.

Support for multiple users and account types.

Real-time transaction history storage.